¿Qué es programar?

Nombre: _____ Fecha: _____

Qué vas a aprender

- Explicar qué es programar, qué es un programa y qué es un algoritmo
- Entender que el ordenador sigue instrucciones exactas, en orden y sin interpretar
- Describir cómo funciona un sistema informático: entrada, proceso, salida y almacenamiento
- Distinguir lenguaje máquina, lenguajes de bloques y lenguajes de texto, y el papel de los traductores
- Reconocer las piezas básicas de un programa y los tipos de errores
- Valorar la programación como herramienta creativa, profesional y responsable

Para pensar

Si le dices a un amigo «tráeme un vaso de agua», lo hace sin problema. ¿Por qué un robot necesitaría muchas más instrucciones para hacer lo mismo?

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Rechazar Aceptar

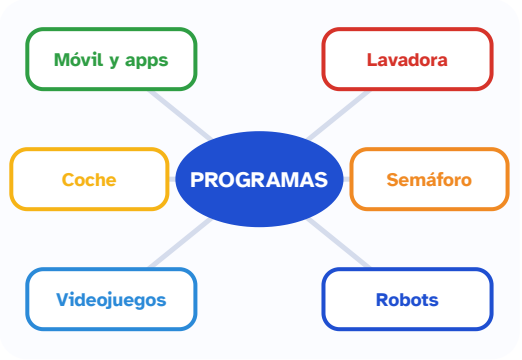
Explicación

1.1 Programar es dar instrucciones

Programar es escribir instrucciones para que una máquina realice una tarea. El conjunto de esas instrucciones, ordenadas y escritas en un lenguaje que la máquina puede entender, se llama programa. A los programas también se les llama **software** o, en el móvil, **aplicaciones**.

Cada vez que usas una aplicación, juegas a un videojuego, pones la lavadora o pasas por un semáforo, se está ejecutando un programa que alguien escribió. Las personas que escriben programas se llaman **programadoras** y **programadores**.

Programar se parece a escribir una receta muy detallada para alguien que no sabe nada de cocina. La diferencia es que el ordenador es **rapidísimo** (hace miles de millones de operaciones por segundo), **nunca se cansa** y repite lo mismo sin equivocarse. Pero si las instrucciones están mal, hará mal la tarea igual de rápido.



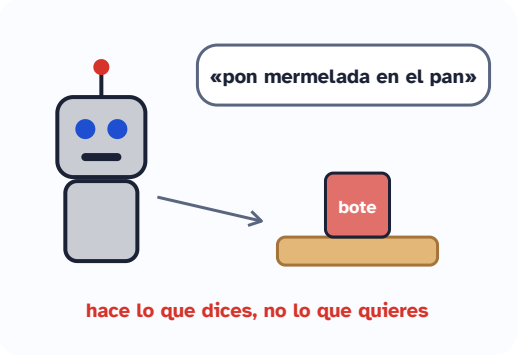
1.2 El ordenador hace exactamente lo que le dices

Un ordenador **no piensa ni adivina**: ejecuta las instrucciones tal como están escritas y en el orden en que aparecen. Por eso las instrucciones tienen que ser precisas, sin ambigüedades.

Una actividad clásica para entenderlo es el «robot sándwich»: un compañero hace de robot y otro le dicta cómo preparar un sándwich. Si le dices «pon la mermelada en el pan», el robot puede poner el bote entero encima de la

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Las personas usamos el **sentido común** para completar lo que falta; las máquinas no. De aquí salen tres reglas de oro: cada instrucción debe ser **clara** (una sola interpretación), las instrucciones deben estar en el **orden correcto** y no puede faltar **ningún paso**. Si cambias el orden de «abrir el grifo» y «poner el vaso», el resultado es un charco.



1.3 Hardware y software: cómo funciona un ordenador

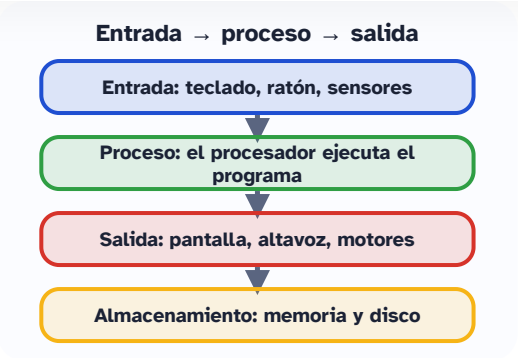
Un sistema informático tiene dos partes. El *hardware* es la parte física, lo que se puede tocar: el procesador, la memoria, la pantalla, el teclado o los motores de un robot. El *software* son los programas: las instrucciones que le dicen al hardware qué hacer. Sin software, el hardware es una caja inútil; sin hardware, el software no tiene dónde ejecutarse.

Casi todos los sistemas siguen el mismo esquema:

- **Entrada**: los datos llegan por el teclado, el ratón, la pantalla táctil, el micrófono o los sensores.
- **Proceso**: el **procesador** (CPU) ejecuta el programa y hace los cálculos.
- **Salida**: los resultados aparecen en la pantalla, suenan por el altavoz o mueven un motor.
- **Almacenamiento**: los programas y los datos se guardan en la memoria y en el disco.

Un cajero automático, una consola o un robot aspirador siguen este esquema: leen datos, los procesan con un programa y producen una respuesta.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

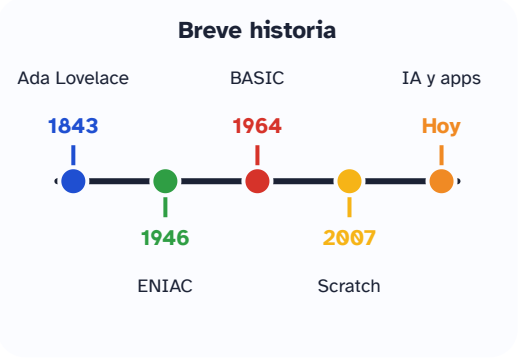


1.4 Un poco de historia

La programación es más antigua que los ordenadores electrónicos. En 1843, la matemática inglesa **Ada Lovelace** escribió un algoritmo para la «máquina analítica» de Charles Babbage, una calculadora mecánica que nunca llegó a construirse. Por eso se la considera la primera programadora de la historia.

En 1946 se presentó el **ENIAC**, uno de los primeros ordenadores electrónicos: ocupaba una sala entera y se programaba conectando cables a mano. Un grupo de seis mujeres matemáticas fueron sus primeras programadoras. En los años cincuenta, **Grace Hopper** creó los primeros traductores, que permitían escribir programas con palabras en lugar de con números.

Después llegaron lenguajes pensados para aprender, como **BASIC** (1964) o **Logo**, con su famosa tortuga que dibujaba. En 2007, el MIT publicó **Scratch**, el lenguaje de bloques que usarás en este curso. Hoy se programa en todas partes: móviles, coches, satélites y sistemas de inteligencia artificial.



1.5 El idioma de la máquina: bits y binario

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Con bits se puede representar cualquier cosa. Los números se escriben en sistema binario, que solo usa ceros y unos: el 0 es 0, el 1 es 1, el 2 es 10, el 3 es 11, el 4 es 100 y el 5 es 101. Las letras también tienen un código: por ejemplo, la «A» es 01000001. Una foto es una lista enorme de números que indican el color de cada punto, y una canción, una lista de números que describen el sonido.

Las instrucciones que entiende directamente el procesador también son ceros y unos: es el lenguaje máquina. Escribir programas así sería lentísimo y muy fácil equivocarse, por eso las personas usamos lenguajes de programación más cercanos a nuestro idioma.

Decimal	Binario	Bits
0	0	1
1	1	1
2	10	2
3	11	2
5	101	3

1.6 Lenguajes de programación: bloques y texto

Un lenguaje de programación es un idioma con palabras y reglas muy estrictas para escribir programas. Igual que en español hay normas de ortografía, cada lenguaje tiene su **sintaxis**: la forma correcta de escribir cada instrucción. El texto de un programa se llama código fuente.

Hay dos grandes familias de lenguajes para aprender:

- **Lenguajes de bloques**, como Scratch, MakeCode o mBlock. Las instrucciones son piezas de colores que encajan como un puzle. No hay errores de escritura y es fácil ver la estructura del programa.
- **Lenguajes de texto**, como Python, JavaScript o C++. Las instrucciones se escriben con el teclado. Son más potentes y se usan en el mundo profesional.

Lo importante es que **las ideas son las mismas**: un bucle «repetir 3 veces» en Scratch es lo mismo que `for i in range(3)` en Python. Lo que aprendas con

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

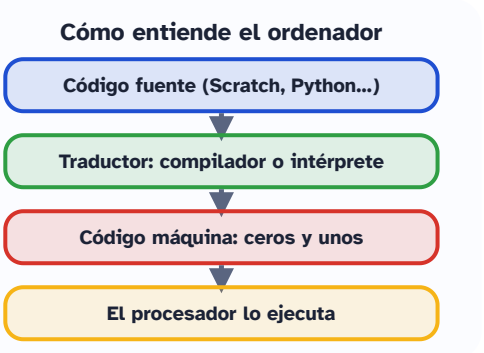


1.7 Los traductores: compiladores e intérpretes

Si el procesador solo entiende ceros y unos, ¿cómo ejecuta un programa escrito en Scratch o en Python? Gracias a un traductor, un programa que convierte el código fuente en lenguaje máquina. Hay dos tipos:

• El compilador traduce el programa **entero** antes de ejecutarlo y crea un archivo ejecutable. Es como traducir un libro completo antes de publicarlo. Así funcionan lenguajes como C o C++. Los programas compilados suelen ser muy rápidos.
• El intérprete traduce y ejecuta el programa **instrucción a instrucción**, mientras funciona. Es como un intérprete que traduce en directo una conversación. Así funcionan Python o Scratch.

Cuando cargas un programa en una placa de robótica, el editor lo compila y lo envía a la placa. Gracias a los traductores, el mismo programa puede funcionar en ordenadores distintos.



1.8 Las piezas de un programa

Aunque haya miles de lenguajes, casi todos los programas se construyen con las

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

• **Secuencia**: instrucciones que se ejecutan una detrás de otra, en orden.
• **Repetición** o **bucle**: un grupo de instrucciones que se repite varias veces o mientras se cumpla algo.
• **Decisión** o **condicional**: el programa elige qué hacer según una condición («si llueve, coge el paraguas»).
• **Datos**: los programas guardan información en **variables**, como los puntos de un juego o el nombre de un usuario.

Además, los programas reaccionan a **eventos** (pulsar una tecla, tocar la pantalla, que un sensor detecte algo) y se organizan en partes más pequeñas llamadas **funciones** o bloques propios. Con estas pocas piezas se construye desde una calculadora hasta un videojuego o el control de un robot. A lo largo del curso aprenderás a usar cada una.



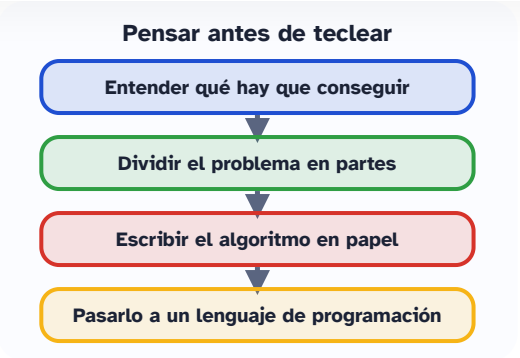
1.9 Pensar antes de teclear

Muchas personas creen que programar es sentarse delante del ordenador y escribir. En realidad, lo más importante pasa **antes**: entender el problema y pensar la solución. Esa forma de pensar se llama pensamiento computacional.

Primero hay que **entender** qué se quiere conseguir: ¿qué datos tengo?, ¿qué resultado espero? Después se **divide** el problema en partes más pequeñas (descomposición): un videojuego se divide en el personaje, los enemigos, los puntos y la pantalla final. Luego se escribe el algoritmo, la lista ordenada de pasos para resolver cada parte, en papel, con frases o con un diagrama de flujo.

Solo entonces se pasa el algoritmo a un lenguaje de programación. Los programadores profesionales dedican mucho tiempo a pensar, dibujar esquemas

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

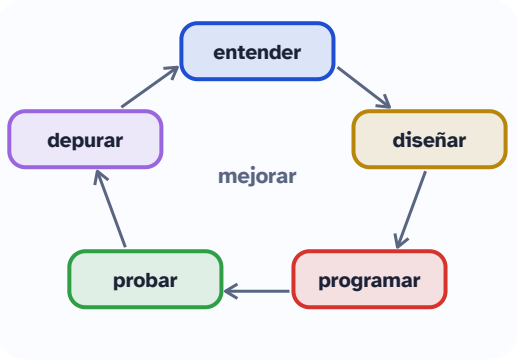


1.10 El ciclo de creación de un programa

Los programas no salen perfectos a la primera. Se crean siguiendo un **ciclo** que se repite varias veces:

1. **Entender** el problema o la idea.
2. **Diseñar** la solución: el algoritmo, los dibujos, los personajes.
3. **Programar**: escribir el código o montar los bloques.
4. **Probar**: ejecutar el programa en situaciones distintas, también las raras.
5. **Depurar**: encontrar y corregir los errores.

Después se vuelve a empezar para mejorar o añadir funciones. Este método de avanzar con versiones pequeñas que se prueban y mejoran se llama **desarrollo iterativo**. Es lo que hacen las empresas de videojuegos o de apps: lanzan una versión, recogen opiniones de los usuarios y publican actualizaciones. Las actualizaciones de tu móvil son justamente eso: nuevas vueltas del ciclo.



1.11 Los bugs y la depuración

Un error en un programa se llama bug («bicho» en inglés). Se cuenta que en

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

bicho encontrado». La tarea de buscar y corregir errores se llama **depurar** (en inglés, *debugging*).

Hay tres tipos de errores:

- **De sintaxis**: la instrucción está mal escrita y el traductor no la entiende, como olvidar un paréntesis en Python. En Scratch casi no ocurren.
- **De ejecución**: el programa empieza, pero falla al funcionar, por ejemplo al dividir entre cero.
- **De lógica**: el programa funciona, pero hace algo distinto de lo que queríamos, como restar puntos en vez de sumarlos. Son los más difíciles de encontrar.

Equivocarse es normal: todos los programadores lo hacen. Lo importante es probar mucho y leer el programa con calma.

Encuentra el bug

```
cuando se pulse la bandera
puntos ← 0
por siempre
    si toca Moneda entonces
        puntos ← puntos − 1 ×
        puntos ← puntos + 1 ✓
# un bug no da error: hace otra cosa
```

1.12 Estrategias para depurar

Encontrar un bug se parece a resolver un misterio. Algunas estrategias útiles:

- **Reproducir el error**: averigua exactamente qué haces para que ocurra.
- **Probar por partes**: ejecuta trozos pequeños del programa por separado para ver cuál falla.
- **Mostrar los valores**: haz que el programa diga en pantalla cuánto valen las variables en cada momento.
- **Hacer una traza**: sigue el programa paso a paso con lápiz y papel, como si fueras el ordenador.
- **Explicarlo en voz alta**: al contarle a otra persona (¡o a un patito de goma!) qué hace cada línea, muchas veces ves el fallo tú mismo.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Un consejo: guarda copias del programa cuando funcione, antes de hacer cambios grandes. Así siempre podrás volver a una versión buena.



1.13 ¿Para qué sirve aprender a programar?

Aprender a programar no es solo para quienes quieren ser informáticos. Programar enseña a **resolver problemas** paso a paso, a ser **preciso**, a **no rendirse** ante los errores y a **crear** cosas propias: un juego, una animación, una web o un robot.

Además, la programación está presente en muchísimas profesiones: diseño de videojuegos, desarrollo de apps y webs, robótica industrial y médica, ciencia de datos e inteligencia artificial, ciberseguridad, investigación del clima o de la genética, arte digital y música electrónica. Incluso médicos, periodistas o arquitectos usan pequeños programas en su trabajo.

Entender cómo funcionan los programas también te convierte en un usuario más **crítico**: sabrás que detrás de cada aplicación hay decisiones tomadas por personas, por ejemplo qué vídeos te recomienda una red social y por qué.

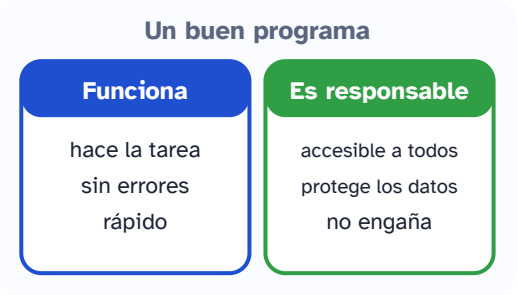


Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Los programas afectan a la vida de millones de personas, así que programar también implica **responsabilidad**. Un buen programa no solo funciona: además es accesible, es decir, lo pueden usar personas con dificultades de visión, de audición o de movimiento (letra grande, buen contraste, subtítulos, control por voz).

También debe **proteger los datos personales** de quien lo usa y no recoger más información de la necesaria. No debe engañar ni enganchar a propósito a los usuarios. Y hay que respetar el trabajo de los demás: si usas código, imágenes o sonidos de otras personas, debes citarlas y respetar su **licencia**.

En la comunidad de Scratch, por ejemplo, se puede hacer una «reinvención» (*remix*) de un proyecto ajeno, pero el autor original aparece siempre reconocido. Muchos programas son **software libre**: su código se comparte para que cualquiera pueda estudiarlo, mejorarlo y compartirlo.



De la idea a la máquina



Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Ejercicios

1. **Define con tus palabras qué es programar y qué es un programa.**

2. **Escribe cinco aparatos de tu casa que funcionen con un programa.**

3. **¿Qué diferencia hay entre hardware y software? Pon dos ejemplos de cada uno.**

4. **¿Qué es un bit? ¿Cuántos bits tiene un byte?**

5. **¿Quién es considerada la primera programadora de la historia y por qué?**

6. **Nombra dos lenguajes de bloques y dos de texto.**

7. **¿Qué es un bug? ¿Y depurar?**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

8. **Indica si es entrada o salida: micrófono, pantalla, teclado, impresora, sensor de luz.**

9. **¿Por qué un ordenador no puede «adivinar» lo que queremos?**

10. **Escribe las cuatro piezas básicas de casi todos los programas.**

11. **Escribe en binario los números 4, 7 y 8.**

12. **¿Qué número decimal es el binario 1011? Explica el cálculo.**

13. **Escribe instrucciones precisas para lavarse los dientes (al menos 7 pasos). ¿Qué pasaría si cambias el orden de dos de ellas?**

14. **Explica la diferencia entre compilador e intérprete con un ejemplo de la vida diaria.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

15. **Un programa de una calculadora muestra 5 al calcular 2 + 2. ¿Qué tipo de error es? ¿Y si el programa no arranca porque falta un paréntesis?**

16. **Describe el esquema entrada-proceso-salida de un cajero automático.**

17. **Pon un ejemplo de instrucción ambigua y reescríbela para que sea precisa.**

18. **Explica por qué se dice que «las ideas de programación son las mismas en bloques y en texto».**

19. **Ordena y explica el ciclo de creación de un programa. ¿Por qué se llama ciclo?**

20. **Busca tres situaciones en las que una persona usa el sentido común y un robot necesitaría instrucciones extra.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

22. **¿Qué significa que un programa sea accesible? Da dos ejemplos.**

23. **Escribe un algoritmo de 6 pasos para que un robot dibuje un cuadrado de 10 cm en un papel.**

24. **Calcula cuántos valores distintos se pueden representar con 1, 2, 3 y 8 bits. ¿Qué regla observas?**

25. **Un programa de un semáforo inteligente cuenta coches con un sensor y alarga el verde si hay muchos. Identifica entrada, proceso, salida y una decisión del programa.**

26. **Escribe en pseudocódigo un programa que pida la edad y diga «puedes entrar» si es mayor o igual que 12 o «lo siento» si no.**

27. **La alarma 1202 del Apolo 11 apareció porque el ordenador recibía demasiadas tareas. Explica por qué el aterrizaje pudo seguir.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

28. **Debate: ¿debería una app pedir acceso a tu ubicación si es un juego de puzles? Razona tu respuesta desde la programación responsable.**

29. **Diseña tres casos de prueba para un programa que dice si un número es par o impar. ¿Por qué conviene probar casos «raros»?**

30. **Explica qué ventajas e inconvenientes tiene un lenguaje de bloques frente a uno de texto.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Practica

1. **Programar es...**

   A) arreglar ordenadores
   B) escribir instrucciones para que una máquina haga una tarea
   C) usar aplicaciones del móvil
   D) diseñar el hardware

2. **Un ordenador ejecuta las instrucciones...**

   A) tal como están escritas y en orden
   B) interpretando lo que queremos
   C) solo si son cortas
   D) en el orden que prefiera

3. **El software es...**

   A) la pantalla y el teclado
   B) el procesador
   C) la carcasa del ordenador
   D) el conjunto de programas

4. **Un sensor de un robot es un elemento de...**

   A) salida
   B) proceso
   C) entrada
   D) almacenamiento

5. **Un bit puede valer...**

   A) de 0 a 9
   B) de 0 a 255
   C) 0 o 1
   D) cualquier letra

6. **Un byte tiene...**

   A) 2 bits
   B) 8 bits
   C) 10 bits

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

A) 101
B) 110
C) 11
D) 100

8. **Se considera la primera programadora a...**
   A) Grace Hopper
   B) Marie Curie
   C) Margaret Hamilton
   D) Ada Lovelace

9. **Scratch es un lenguaje...**
   A) máquina
   B) de texto profesional
   C) de compilación
   D) de bloques

10. **El texto de un programa escrito por una persona se llama...**
    A) código máquina
    B) hardware
    C) código fuente
    D) byte

11. **El traductor que traduce todo el programa antes de ejecutarlo es el...**
    A) intérprete
    B) compilador
    C) procesador
    D) editor

12. **El traductor que va traduciendo instrucción a instrucción es el...**
    A) intérprete
    B) compilador
    C) sistema binario
    D) sensor

13. **Un error que hace que el programa funcione pero haga otra cosa es de...**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

C) hardware
D) compilación

14. **Olvidar un paréntesis en Python es un error de...**

A) lógica
B) ejecución
C) diseño
D) sintaxis

15. **Buscar y corregir errores se llama...**

A) compilar
B) ejecutar
C) depurar
D) remezclar

16. **La fase que va justo antes de programar es...**

A) depurar
B) diseñar el algoritmo
C) probar
D) publicar

17. **Repetir un grupo de instrucciones es un...**

A) condicional
B) bucle
C) evento
D) bit

18. **«Si llueve, coge el paraguas» es un ejemplo de...**

A) decisión o condicional
B) bucle
C) variable
D) secuencia

19. **Un programa accesible es el que...**

A) es gratis
B) no tiene errores
C) solo funciona en móviles

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

A) borra el proyecto original

B) está prohibida

C) reconoce siempre al autor original

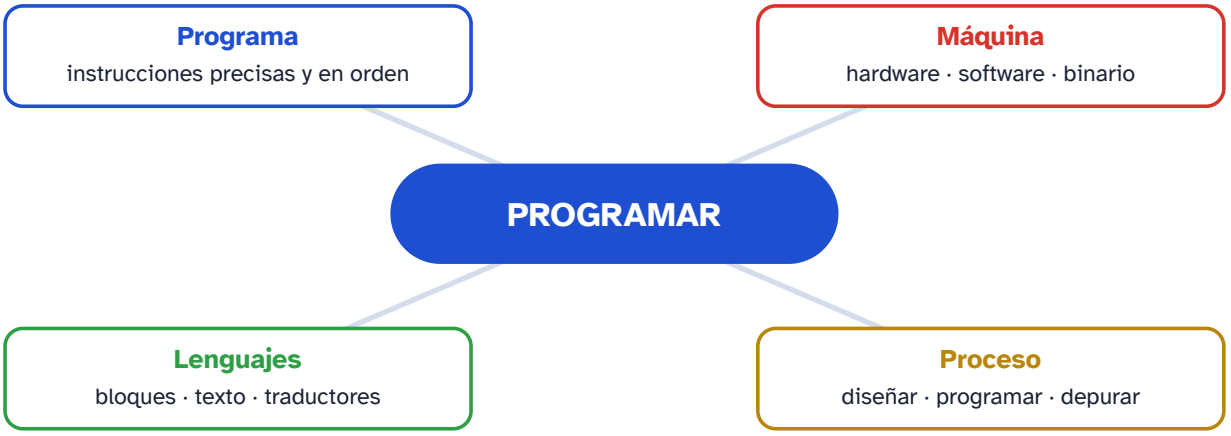
D) no se puede compartir

Explícalo con tus palabras

Incluye: Qué es programar y por qué el ordenador necesita instrucciones precisas; Cómo funciona un sistema informático: entrada, proceso y salida; Por qué el ordenador usa ceros y unos y para qué sirven los traductores; Qué tipos de errores hay y cómo se depuran; Por qué programar es útil aunque no quieras ser informático.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Resumen



Glosario

Programar	Escribir instrucciones ordenadas para que una máquina realice una tarea.
Programa	Conjunto de instrucciones escritas en un lenguaje de programación que una máquina puede ejecutar.
Algoritmo	Secuencia ordenada y finita de pasos que resuelve un problema.
Hardware	Componentes físicos de un sistema informático: procesador, memoria, pantalla, sensores...
Software	Programas y datos que hacen funcionar el hardware.
Procesador (CPU)	Componente que ejecuta las instrucciones de los programas y hace los cálculos.
Bit	Dígito binario, la unidad mínima de información; vale 0 o 1.
Sistema binario	Sistema de numeración que solo usa los dígitos 0 y 1.
Lenguaje máquina	Instrucciones en código binario que el procesador entiende directamente.
Lenguaje de programación	Idioma con palabras y reglas estrictas para escribir

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Código fuente	Texto o bloques de un programa tal como lo escribe el programador.
Compilador	Programa que traduce el código fuente entero a lenguaje máquina antes de ejecutarlo.
Intérprete	Programa que traduce y ejecuta el código instrucción a instrucción.
Bug	Error en un programa que hace que falle o funcione de forma distinta a la deseada.
Depurar	Buscar, entender y corregir los errores de un programa.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Programar

Escribir instrucciones para que una máquina haga una tarea.

Programa

Conjunto ordenado de instrucciones en un lenguaje que la máquina entiende.

Hardware

Parte física de un sistema informático.

Software

Los programas que dicen al hardware qué hacer.

Entrada · proceso · salida

Datos que llegan, cálculo del procesador y resultados.

Bit

Unidad mínima de información: 0 o 1. 8 bits = 1 byte.

Lenguaje máquina

Instrucciones en ceros y unos que entiende el procesador.

Código fuente

Programa escrito por una persona en un lenguaje de programación.

Compilador

Traduce todo el programa antes de ejecutarlo.

Intérprete

Traduce y ejecuta instrucción a instrucción.

Bug

Error de un programa: de sintaxis, de ejecución o de lógica.

Depurar

Buscar y corregir los errores de un programa.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Soluciones

1. **B)** Un programa es un conjunto ordenado de instrucciones.
2. **A)** No adivina ni usa el sentido común.
3. **D)** El hardware es la parte física.
4. **C)** Lleva información del entorno al sistema.
5. **C)** Dos estados: hay corriente o no.
6. **B)** 8 bits forman un byte.
7. **A)** 4 + 0 + 1.
8. **D)** Escribió un algoritmo en 1843.
9. **D)** Bloques de colores que encajan.
10. **C)** Luego se traduce a código máquina.
11. **B)** Crea un ejecutable.
12. **A)** Como un intérprete en directo.
13. **A)** Son los más difíciles de encontrar.
14. **D)** El código está mal escrito.
15. **C)** En inglés, debugging.
16. **B)** Primero se piensa la solución.
17. **B)** También se llama repetición.
18. **A)** Se elige según una condición.
19. **D)** Subtítulos, contraste, voz...
20. **C)** Respeto a la autoría.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Soluciones de los ejercicios

1. Programar es escribir instrucciones para que una máquina haga una tarea. Un programa es el conjunto ordenado de esas instrucciones escrito en un lenguaje que la máquina puede ejecutar.
2. Por ejemplo: el móvil, la lavadora, el microondas, la televisión y el router. Todos tienen un procesador que ejecuta instrucciones.
3. El hardware es la parte física (teclado, procesador); el software son los programas (un navegador, un juego). El hardware necesita software para saber qué hacer.
4. Un bit es la unidad mínima de información: vale 0 o 1. Un byte tiene 8 bits.
5. Ada Lovelace, porque en 1843 escribió un algoritmo para la máquina analítica de Babbage, pensado para que lo ejecutara una máquina.
6. Bloques: Scratch y MakeCode (o mBlock). Texto: Python y JavaScript (o C++).
7. Un bug es un error en un programa. Depurar es buscar y corregir esos errores.
8. Entrada: micrófono, teclado y sensor de luz (llevan datos al sistema). Salida: pantalla e impresora (muestran resultados).
9. Porque no tiene sentido común ni entiende el contexto: solo ejecuta las instrucciones que tiene, tal como están escritas y en orden.
10. Secuencia (pasos en orden), repetición (bucles), decisión (condicionales) y datos (variables).
11. Con posiciones 8, 4, 2, 1: el 4 es 100, el 7 es 111 (4 + 2 + 1) y el 8 es 1000.
12. Posiciones 8, 4, 2, 1: 1·8 + 0·4 + 1·2 + 1·1 = 8 + 2 + 1 = 11.
13. Coger el cepillo, abrir el grifo, mojarlo, cerrar el grifo, poner pasta, cepillar dos minutos, enjuagar la boca, aclarar el cepillo y guardarlo. Si pones «cepillar» antes que «poner pasta», el cepillado no sirve: el orden importa.
14. El compilador traduce todo el programa antes de ejecutarlo, como quien traduce un libro entero antes de publicarlo. El intérprete traduce y ejecuta instrucción a instrucción, como un intérprete en una conversación en directo.
15. El primero es de lógica (funciona pero da un resultado incorrecto). El segundo es de sintaxis (el código está mal escrito y el traductor no lo entiende).
16. Entrada: tarjeta, teclado con el PIN y la cantidad. Proceso: el procesador comprueba el PIN y el saldo. Salida: billetes, recibo y mensajes en la pantalla. Almacenamiento: los datos de la operación.
17. «Echa un poco de sal» es ambigua. Precisa: «echa media cucharadita de sal en la olla».
18. Porque secuencias, bucles, condicionales y variables existen en todos los lenguajes; solo cambia cómo se escriben. Un «repetir 3» de Scratch equivale a un for de Python.
19. Entender, diseñar, programar, probar y depurar. Se llama ciclo porque después se vuelve a empezar para mejorar el programa o añadir funciones (desarrollo iterativo).
20. Por ejemplo: saber que un vaso está lleno sin que nadie lo diga, esquivar a una persona

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

21. Por ejemplo: mostrar en pantalla el valor de las variables para ver cuándo cambian mal, y hacer una traza paso a paso en papel. También probar el programa por partes.
22. Que lo puedan usar personas con distintas capacidades. Ejemplos: subtítulos en los vídeos para personas sordas y buen contraste o lectura en voz alta para personas con baja visión.
23. Bajar el rotulador; avanzar 10 cm; girar 90° a la derecha; repetir avanzar y girar hasta completar 4 lados; subir el rotulador. Con un bucle: repetir 4 veces [avanzar 10, girar 90].
24. 1 bit: 2; 2 bits: 4; 3 bits: 8; 8 bits: 256. Cada bit más duplica el número de valores: con n bits hay 2^n valores.
25. Entrada: el sensor que cuenta coches. Proceso: el controlador compara el número de coches con un límite. Salida: las luces. Decisión: si hay más coches que el límite, alargar el verde; si no, tiempo normal.
26. preguntar edad; si edad ≥ 12 entonces decir «puedes entrar»; si no, decir «lo siento». Usa entrada, decisión y salida.
27. Porque el programa de Margaret Hamilton ordenaba las tareas por prioridad: si había demasiadas, descartaba las menos importantes y mantenía las vitales, como controlar los motores. Era un programa preparado para los errores.
28. No: un buen programa solo recoge los datos que necesita. Un juego de puzles no necesita la ubicación; pedirla sin motivo pone en riesgo la privacidad. Se puede rechazar el permiso y el juego debería funcionar igual.
29. Probar 4 (par), 7 (impar) y 0 (par; caso límite). También un número negativo, como –3. Los casos raros o límite son donde más se esconden los bugs.
30. Ventajas de los bloques: no hay errores de sintaxis, se ve la estructura y es fácil empezar. Inconvenientes: programas grandes ocupan mucho espacio, son más lentos de montar y no se usan tanto en el mundo profesional, donde predominan los lenguajes de texto, más potentes y flexibles.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información